



COM 11117 - Introducción al Desarrollo Web

Segundo proyecto integrador

Primavera 2026

El presente proyecto tiene como objetivo integrar los conocimientos adquiridos en las áreas de HTML, CSS, Bootstrap, JavaScript, Git/GitHub, consumo de APIs e implementación con Python, React con Vite, Bases de datos y despliegue de aplicaciones mediante el desarrollo de una aplicación web. También busca fomentar el uso práctico de las herramientas estudiadas en clase y promover el trabajo colaborativo con compañeros y amigos.

Consideraciones generales

El trabajo se realizará en equipos de máximo cuatro integrantes, fomentando la colaboración, la división de responsabilidades técnicas y la documentación clara del proceso de desarrollo.

Cada equipo deberá elegir uno de los tópicos propuestos, aplicando los mismos principios técnicos y buenas prácticas, pero adaptándolos a un contexto temático distinto.

- Responsivo: Adaptable a dispositivos móviles y de escritorio usando Bootstrap.
- Consumo de API: Obtener datos desde una API propia usando fetch:
 - GET/POST/PATCH/PUT/DELETE
- Consumo de una API de terceros (dependiendo el caso) desde el backend con la API propia de modo que el resultado se entregue transformado al llamar a la API.
- Uso de headers para identificar al usuario que haga la llamada a la API y extraer la información correcta por medio de columnas en la base de datos.
- Implementación del protocolo OpenGraph para compartir su página web por medio de redes sociales (incluyendo WhatsApp): <https://ogp.me/>
- Documentación en GitHub

Consideraciones técnicas

- Aplicar los fundamentos de React junto con los de HTML para mantener una semántica y estructura correcta.
- Implementar diseños responsivos con Bootstrap 5.3.
- Uso de funciones, hooks de estado y de efecto dentro del frontend.
- Integrar OpenGraph en su página web.

- Usar Git y GitHub como control de versiones colaborativo. Es obligatorio el uso de Pull requests (PR) y solo puede haber un commit inicial en la rama main. Los demás se deben incorporar por medio del merge del PR.
- Desplegar la página web de React y desplegar la API para consumo de información. Es obligatorio (no es necesario dominio personalizado) y es obligatorio el uso de base de datos en PostgreSQL.

Tópicos para escoger

Imitación de la página de la FIFA

- Tomar la página de la [Copa Mundial de la FIFA 2026™](#) como referencia y adaptarla de modo que TODA la información se obtenga de tu base de datos. Por lo menos debes de tener en tu versión “Historias destacadas”, “Clasificación”, “El escenario de la Copa Mundial está listo”, “Ciudades Anfitrionas”, “Feed Instagram”, “Países Anfitriones”.
- Formulario para crear “Historias destacadas” con url de imagen, título, sección y cuerpo de la noticia. Asimismo, debe existir la opción de editar dichas “Historias destacadas” y también la opción de eliminarlos.
- Formularios para editar, reemplazar y borrar.
- Mostrar un apartado personal en el que se haga llamada a la API propia (en Python) y que está a su vez haga una llamada a la API de API-FOOTBALL para mostrar lo nuevo en la plataforma: [API-Football - Restful API for Football data](#). Puedes usar el endpoint <https://www.api-football.com/documentation-v3#tag/Players/operation/get-players-topscorers> o algún otro de tu interés.
- La primera vez que se haga el render del feed, debe de llamar a la base de datos para mostrar las “Historias destacadas” y estas se deben guardar en local storage, así como el timestamp del consumo de la info. En las siguientes ocasiones si estos campos no son vacíos, se debe de llamar a la API, pero pasando el parámetro de fecha mínima.

Imitación de Pinterest

- Interfaz con el mosaico de las diferentes imágenes/posts.
- Formulario de alta (usuario, link a imagen, fecha de alta, etiquetas).
- Formularios para editar, reemplazar y borrar.
- Mostrar un apartado de descubrimiento en el que se haga llamada a la API propia (en Python) y que está a su vez haga una llamada a la API de Unsplash para mostrar imágenes aleatorias <https://unsplash.com/documentation#list-photos>. Está lista no

debe ser regresada tal cual, sino que debe ser transformada en la API para sólo regresar lo necesario para hacer el render de la foto.

- La primera vez que se haga el render del feed, debe de llamar a la base de datos para mostrar los posts y estos se deben guardar en local storage, así como el timestamp del consumo de la info. En las siguientes ocasiones si estos campos no son vacíos, se debe de llamar a la API, pero pasando el parámetro de fecha mínima.

Imitación del periódico New York Times

- Interfaz con el listado de las noticias publicadas por el periódico (debe de tener un botón para cambiar la vista entre cards y un listado de la información con la fotografía a la izquierda).
- Formulario de alta (título, link a imagen, fecha de alta, texto de la noticia).
- Formularios para editar, reemplazar y borrar.
- Mostrar un apartado de trending topics en el que se haga llamada a la API propia (en Python) y que está a su vez haga una llamada a la API de <https://developer.nytimes.com/apis> para mostrar las noticias más relevantes. Esta lista no debe ser regresada tal cual, sino que debe ser transformada en la API para sólo regresar lo necesario para hacer el render.

Todos los proyectos

Independientemente de lo mencionado anteriormente deben estar presentes los

siguientes elementos:

- Endpoint con listado de entidades que use paginación.
- Endpoint para obtener una sola entidad por id (debes buscar la forma en presentarlo en la interfaz gráfica).
- Endpoint para dar de alta una nueva entidad (debes buscar la forma en presentarlo en la interfaz gráfica).
- Endpoint para modificar una nueva entidad (debes buscar la forma en presentarlo en la interfaz gráfica).
- Endpoint para eliminar una nueva entidad (debes buscar la forma en presentarlo en la interfaz gráfica).
- No deberán de poder modificarse o eliminar entidades si el usuario que las creó no es el mismo que hace estas peticiones (cuando tengan la página generen un formulario para guardar en sessionStorage el usuario actual, y luego usen este valor para pasarlo en el header).
- Uso de paginación del lado del cliente.

- No copien las páginas mencionadas, y mucho menos los colores de marca. Traten de darle su toque original y que sea visualmente consistente.
- Uso de OpenGraph para añadir título, descripción, imagen principal, etc. para compartir su página en redes sociales de forma correcta.

Entregables

Repositorio en GitHub que incluya código funcional para el frontend y el backend.

- README.me con las siguientes secciones:
 - Resumen del producto.
 - Instrucciones de cómo levantar el frontend.
 - Instrucciones de cómo levantar el backend. Es indispensable incluir un requirements.txt y el comando para levantar el servidor.
- Enlace a tu página web del cliente.
- Enlace a tu health endpoint principal de la API (el health endpoint debe tener también un apartado que corrobore que la llamada a la API externa está activa)
- Sección de autores con nombre de pila, fotografía no requerida.
- Live demo de su proyecto.
- Resolución de dudas del profesor posterior a la presentación